ASSIGNMENT 3 – TASK 3

SEGMENTATION: ADDRESS TRANSLATION, PROTECTION AND FRAGMENTATION

1. Introduction and Problem Context

The assignment describes student programs whose code should be read-only and shareable, whose stack grows, and whose data region varies in size. Segmentation represents a program using logical regions such as code, data and stack. Each segment has a base address and a limit, enabling segment-level bounds checks and protection. This report follows Q3 – Segmentation (6 Marks, K5 – Evaluate), on pages 25–26 of the supplied question paper.

2. Segmentation and the Programmer's View

A segmented address space is divided into variable-sized logical units. A program can refer to a code segment, data segment or stack segment, rather than treating its entire address space as one undifferentiated sequence. The segment table stores the base and limit for each segment. The base identifies where the segment begins; the limit gives the maximum permitted offset.

Segment	Typical contents	Protection / sharing example
Code	Executable instructions and constants.	Read + execute, not write; identical code can be shared by programs.
Data	Global, static and dynamically allocated data.	Read/write; normally private unless explicitly shared.
Stack	Function frames, local variables and return information.	Read/write; private to each process and may grow within its allowed range.

This logical division matches how programmers and compilers organise a program. Different segments can have different sizes and access permissions. A segment can be shared by mapping multiple processes to the same region, provided protection permits the intended accesses.

3. Segment-Offset Address Translation

For logical address (segment, offset), the processor selects a segment-table entry and checks the offset against the segment limit. If valid, the address is:

Address = Segment Base + Offset, provided Offset ≤ Limit

For this demonstration, the segment table uses CODE base 0x10000/limit 0x0FFF, DATA base 0x30000/limit 0x07FF and STACK base 0x70000/limit 0x03FF. The limit is inclusive. In pure segmentation without paging, base-plus-offset is treated as the physical address for this exercise; with paging enabled, it is a linear address requiring a second translation.

3.1 Eight address calculations

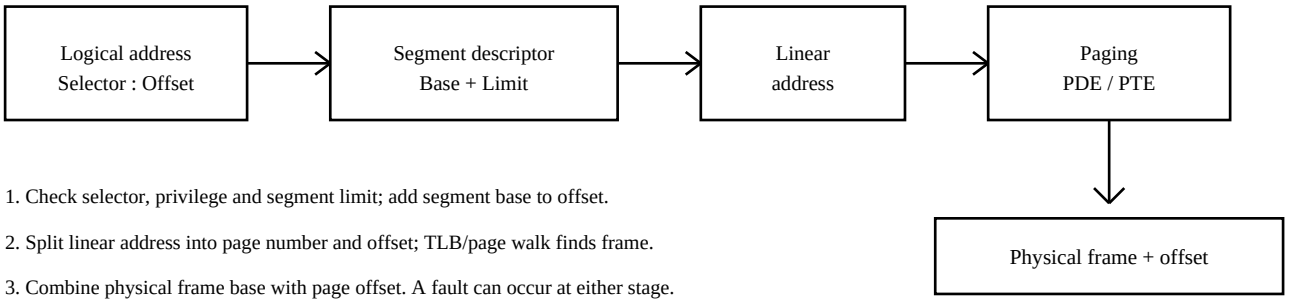
No.	Segment:offset	Base	Limit check	Result
1	CODE:0000	0x10000	0 ≤ 0x0FFF	0x10000
2	CODE:0123	0x10000	0x0123 ≤ 0x0FFF	0x10123
3	CODE:0FFF	0x10000	0x0FFF ≤ 0x0FFF	0x10FFF
4	CODE:1000	0x10000	0x1000 > 0x0FFF	Limit fault
5	DATA:0040	0x30000	0x0040 ≤ 0x07FF	0x30040
6	DATA:0800	0x30000	0x0800 > 0x07FF	Limit fault
7	STACK:03FF	0x70000	0x03FF ≤ 0x03FF	0x703FF
8	STACK:0400	0x70000	0x0400 > 0x03FF	Limit fault

segment_translation.c verifies the same eight cases. It prints the base-plus-offset result for valid addresses and reports a segmentation fault when the offset exceeds the segment limit.

4. Segmentation with Paging – Intel x86 Address Translation

In the classic Intel IA-32 protected-mode model, a logical address contains a segment selector and offset. The selector identifies a descriptor in the Global Descriptor Table (GDT) or Local Descriptor Table (LDT). The descriptor supplies the segment base, limit and access rights. After descriptor and limit/privilege checks, the processor adds base to offset to form a linear address. Paging then translates the linear address to a physical address using page-directory and page-table entries; the TLB caches recent translations.

Segmentation with paging – classic IA-32 protected-mode path



1. Check selector, privilege and segment limit; add segment base to offset.
2. Split linear address into page number and offset; TLB/page walk finds frame.
3. Combine physical frame base with page offset. A fault can occur at either stage.

segmentation_paging_demo.c illustrates this two-stage path: selector/segment plus offset gives a linear address; the linear page number is looked up in a simplified page map; the frame number is combined with the page offset. Its array is an educational model, not an implementation of x86 hardware page-directory structures. The diagram describes classic IA-32; modern x86-64 long mode uses a mostly flat segmentation model, with limited segment-base behaviour such as FS/GS still relevant.

5. Segment Protection and Sharing

Segment descriptors can express access rights such as readable, writable and executable, together with privilege information. A code segment can be read/execute but not writable, preventing ordinary stores from changing instructions. Data and stack segments are typically readable and writable. The processor

checks access rights and bounds before allowing an access.

Sharing is possible when multiple processes' segment descriptors refer to the same code region. Read-only code is a natural candidate because processes cannot modify shared instructions. Writable data and stacks are usually private; explicitly shared writable data requires synchronisation as well as suitable mappings and permissions.

6. External Fragmentation in Pure Segmentation

Segments have variable sizes and each segment normally needs a contiguous region. As segments are loaded and removed, free memory is split into holes. Total free space can be sufficient for a requested segment while the largest individual hole is too small. This is external fragmentation. Compaction can combine holes but requires moving segments and adjusting their bases; paging avoids physically contiguous page placement.

6.1 Experiment: load and remove 20 random-size segments

segment_fragmentation.c models 1,024 KiB of memory. It generates 20 deterministic pseudo-random segment sizes from 12 KiB to 72 KiB, loads each using first-fit placement, then removes every second segment. It measures total free memory, the largest free hole, the number of holes and Total Free − Largest Hole.

Measure after removal	Result
Segments initially loaded	20
Segments removed	10 (odd-numbered IDs)
Total free memory	647 KiB
Largest free hole	297 KiB
Number of separate holes	10
External-fragmentation indicator	647 – 297 = 350 KiB
Test request	298 KiB: total free is sufficient, but no hole can fit it.

The test request is 1 KiB larger than the largest hole. Since 298 KiB is less than total free (647 KiB) but greater than the largest contiguous hole (297 KiB), it cannot be placed. This demonstrates external fragmentation. Results are deterministic for the program's fixed seed and allocation policy.

7. Linux Process Segments and /proc//maps

Linux exposes a process's virtual memory areas (VMAs) through /proc//maps. The map commonly shows executable file-backed regions with read/execute permissions, read-only mappings, writable data, the heap ([heap]), shared libraries, anonymous mappings and the stack ([stack]). Permission fields such as r-xp or rw-p indicate access and mapping status.

segment_maps.c prints its PID and reads /proc/self/maps. The run script also holds the program open briefly, obtains its PID, and captures /proc//maps directly. These Linux VMAs are operating-system memory mappings; they should not be mistaken for classic CODE/DATA/STACK hardware segment descriptors.

8. Segmentation Fault Experiment

segfault_demo.c forks a child and performs an intentional null-pointer write only in the child. The parent waits and checks whether the child ended due to SIGSEGV. This isolates the fault so the test driver can continue.

Expected observation: the child terminates by SIGSEGV (signal 11 on the tested Linux environment), while the parent reports that the OS detected an invalid virtual-memory access. On modern x86, a null-pointer access normally causes a page-fault exception because virtual page zero is not mapped; Linux then delivers SIGSEGV. This is commonly called a segmentation fault, but it is not necessarily a classic segment-limit violation. A segment-limit violation in classic segmentation is a different protection exception.

9. Compare Pure Paging, Pure Segmentation and Segmented Paging

Aspect	Pure paging	Pure segmentation	Segmentation with paging
Logical view	Fixed-size pages; not inherently aligned with program regions.	Variable-size code, data and stack segments.	Logical segments remain visible, but each segment is paged.
Physical allocation	Pages can be placed in any free frames.	Each segment needs a contiguous region.	Pages of each segment can occupy non-contiguous frames.
Fragmentation	No external fragmentation; internal waste in final page.	External fragmentation; little or no page-tail waste.	No external fragmentation at page-frame level; page-tail internal waste remains.
Protection/sharing	Per-page permissions; share selected pages.	Per-segment bounds and access rights; share segments.	Segment and page permissions; share selected pages.
Translation	Page number → PTE/frame + offset.	Check limit; base + offset.	Segment check/base addition, then page translation.

10. Compare Table Sizes – Explicit Example

The assignment does not specify address width, descriptor size, page size or page-table-entry width. This illustrative comparison assumes a 32-bit virtual address, 4 KiB pages, 4-byte page-table entries, three segments (code 12 KiB, data 8 KiB, stack 4 KiB) and 8-byte segment descriptors (32-bit base plus 32-bit limit). These segments occupy six 4 KiB pages. Paging metadata counts leaf entries for these mapped pages in the sparse example; upper-level directory overhead is excluded.

Scheme	Illustrative table requirement	Approximate size
Pure segmentation	3 segment descriptors × 8 bytes	24 bytes
Pure paging – flat 32-bit table	2^20 virtual pages × 4-byte PTE	4 MiB theoretical flat table
Pure paging – six mapped leaf entries	6 × 4-byte PTE, excluding hierarchy	24 bytes
Segmented paging	24-byte descriptors + 6 × 4-byte PTEs	48 bytes, excluding upper levels

Pure segmentation needs a small descriptor table for a few logical regions but suffers external fragmentation. A flat paging table can be large for a wide address space, although hierarchical paging allocates lower-level tables only for used regions. Segmented paging combines segment descriptors with

page tables for pages within those segments; real hardware structures have additional directory and table levels.

11. Alternative Concept: Use Pure Paging Instead of Segmentation

With pure paging, code, data and stack are represented as pages in one virtual address space rather than separate variable-sized segments. Each page-table entry carries the frame mapping and protection bits. Code pages can be read/execute and shared; data and stack pages can be private and writable. Copy-on-write can keep pages shared until a process modifies them.

Pure paging removes external fragmentation because each virtual page can map to any available physical frame. It retains internal fragmentation in the final page. Translation uses a TLB and, on a miss, a hardware page-table walk. Table space depends on mapped virtual pages and the paging hierarchy. Code sharing and protection are straightforward through shared read-only mappings and per-page permissions, but the programmer's explicit code/data/stack abstraction is less directly represented.

12. Why Modern Operating Systems Rely Mainly on Paging

Paging suits virtual memory, demand loading, swapping, copy-on-write, memory protection and sharing because the OS can map fixed-size pages independently. It avoids external fragmentation in physical allocation and works naturally with page faults and hardware TLBs. Modern x86-64 systems generally use a flat segmentation model and rely on paging for fine-grained virtual-memory management. Segmentation remains a useful logical protection and organisation concept, but variable-sized contiguous allocation is harder to scale.

13. Conclusion

Segmentation models a program as logical code, data and stack regions with separate base, limit and access permissions. The eight address calculations show successful translations and limit violations. The x86-style demonstration shows segmentation producing a linear address before paging maps it to a physical frame. The 20-segment experiment demonstrates external fragmentation: 647 KiB is free, but the largest hole is 297 KiB, so a 298 KiB segment cannot be loaded. Linux /proc//maps provides the real process VMA view, and the SIGSEGV experiment explains OS handling of an invalid access. Pure paging avoids external fragmentation and supports virtual memory, while segmented paging combines logical descriptors with page-granular physical allocation.

14. Execution Procedure and Evidence

1. Place segment_translation.c, segmentation_paging_demo.c, segment_maps.c, segfault_demo.c, segment_fragmentation.c and run_task3.sh in one Task03 folder.
2. Open Ubuntu/WSL in that directory.
3. Run chmod +x run_task3.sh.
4. Run ./run_task3.sh.
5. Verify all eight segment-offset results and the limit-fault cases.
6. Review segmentation_paging_output.txt and segment_maps_pid_maps.txt.
7. Verify the child SIGSEGV report and fragmentation results.

8. Capture screenshots of the terminal outputs and preserve the generated text files.

Evidence checklist: eight manual address calculations verified by C; segment-limit fault; segmentation-with-paging diagram and demonstration; live /proc//maps output; controlled SIGSEGV experiment; 20-segment fragmentation calculation; comparison of pure paging, pure segmentation and segmented paging; and table-size comparison.

Source alignment: Q3 – Segmentation, 6 Marks, K5 – Evaluate, pages 25–26 of the supplied Assignment 3 question paper.